기존 JSP 프로젝트에 Vue 점진 도입 사례

> 기존 서버 렌더링 기반 프로젝트를 전면 교체하지 않고,
> 별도 Vue 프로젝트를 결합하는 방식으로 점진 도입한 경험을 정리

소개

기존 aFrame / JSP 기반 프로젝트에 Vue.js를 별도 프론트엔드 레이어로 추가한 사례를 정리한 문서이다. 기존 서버 렌더링 구조를 전면 교체하는 것이 아니라, **기존 프로젝트를 유지한 채 Vue 기반 화면과 라우팅 구조를 점진적으로 적용**하는 것이다. 기존 전자정부 프로젝트에 Vue 프레임워크를 적용해 Java 서비스를 호출하는 방법을 다루고 있다.

배경

JSP 구조는 기능별 화면을 빠르게 구현하는 데에는 익숙했지만, 상호작용이 많은 화면이나 재사용 가능한 UI 구조를 만들기에는 제약이 있었다. 화면 전환과 사용자 인터랙션을 더 유연하게 만들기 위한 목적에서 Vue를 도입했다고 볼 수 있다.

버전 선택의 경우 개발 시점(23.02.22) 기준 Vue는 2와 3이 공존하는 과도기였으며, Vue3의 **안정성 문제, 지원 라이브러리, Internet Explorer 미지원**을 이유로 Vue 2.6.14를 선택했다. 따라서 기존 **시스템 호환성과 안정성을 고려**해 선정했다.

적용 방법

이 작업의 중점은 기존 프로젝트를 유지한 채, Vue 프로젝트를 추가한 점이다. Vue 프로젝트는 별도 디렉터리에서 관리하고, Vue 프로젝트를 생성했다. 운영 환경에서는 시작 페이지가 이 경로를 바라보도록 연결해, 사용자가 시스템에 진입하면 Vue 화면이 자연스럽게 시작되도록 구성했다.

그로인해 백엔드 구조, WAS, 기존 서비스 호출 방식은 유지하면서도 프론트엔드 화면 계층만 점진적으로 바꿀 수 있었다. 이 구조는 기존 JSP 프로젝트를 전면 재구성하지 않고도 Vue 화면을 단계적으로 붙일 수 있는 이점이 있다.

구조

화면 코드는 애플리케이션 디렉터리 아래에서 관리하고, 정적 자산은 별도 자산 디렉터리, 화면 이동은 라우터, 전역 상태는 상태관리 디렉터리에서 담당하도록 구성했다. 여기에 빌드 산출물만 기존 웹 애플리케이션의 배포 경로로 연결해, 레거시 프로젝트와 프론트엔드 프로젝트의 경계를 느슨하게 분리했다.

이 구조의 장점은 다음과 같다. 기존 JSP 프로젝트의 백엔드 구조는 유지하면서도, Vue 쪽에서는 컴포넌트 단위 분리와 라우팅 구조를 따로 가져갈 수 있다. 즉, **레거시 서버 구조와 현대적인 프론트엔드 구조를 느슨하게 결합한 형태**라고 볼 수 있다.

동작 흐름

사용자가 시스템에 진입하면 기존 시작 페이지가 **Vue 빌드 산출물 경로**로 연결된다. 이후 먼저 **최상위 레이아웃**이 로드되고, 그 안의 **콘텐츠 라우팅 영역**에서 현재 경로에 맞는 화면이 렌더링된다. 기본 경로는 **로그인 화면**에 연결되어 있어 첫 진입 시 로그인 화면이 출력되고, 이후 라우팅 설정에 따라 메인 화면과 상세 화면으로 이동한다. 라우터 구성은 `/`를 로그인 화면, `/main`을 메인 화면, `/detail`을 상세 화면에 연결하는 구조다.

로그인 화면에서는 입력값을 `v-model`로 양방향 바인딩하고, 버튼 클릭 또는 Enter 입력 시 로그인 메서드를 호출한다. 이후 axios POST 요청으로 기존 Java 서비스를 호출하고, 응답 결과에 따라 라우터를 통해 다음 화면으로 이동하는 구조다.

이 흐름은 기존 JSP의 form submit 방식과 달리, 화면 렌더링과 데이터 요청을 프론트엔드에서 더 세밀하게 제어할 수 있다는 이점이 있다.

고려사항

이 사례에서 중요했던 것은 **기존 프로젝트와의 공존 방식**이었다.

첫째, 형상관리 대상을 분리했다. 의존성 디렉터리는 `package.json`을 기준으로 다시 설치할 수 있고, 빌드 산출물은 파일명이 매번 달라질 수 있기 때문에, 커밋 대상에서 제외하는 방식으로 관리했다. 이렇게 해야 기존 프로젝트에 Vue를 합친 뒤에도 저장소가 불필요하게 비대해지지 않는다.

둘째, 개발 환경과 실행 환경을 분리했다. 개발 중에는 Vue 내장 서버를 사용해 실시간 반영을 확인하고, 실제 애플리케이션 실행은 빌드된 정적 파일을 Tomcat이 제공하는 구조로 가져갔다. 즉, 개발 편의성은 유지하면서 운영 환경과 연결되는 경로는 기존 시스템 안에 두는 절충안을 선택했다.

한계와 개선 방향

기존 JSP 프로젝트에 Vue를 빠르게 적용하는 데에는 효과적이지만, 몇 가지 한계도 있다.

첫째, 장기적으로는 Vue2 기반이라는 점이 한계다. Vue3로의 완전 전환 이후면 Vue2와 문법 차이로 인해 이행 문제가 발생할 수 있다. 즉, 당시에는 현실적인 선택이었지만, 장기적으로는 Vue3 전환 전략 전략이 필요하다.

둘째, 기존 JSP와 Vue가 함께 존재하는 구조이기 때문에 자산 경로, 라우팅 규칙, 배포 흐름을 계속 일관되게 관리해야 한다. 기존 서버 렌더링 구조와 Vue 기반 라우팅 구조가 함께 존재하기 때문에, 자산 경로, 라우팅 규칙, 배포 흐름을 계속 일관되게 관리해야 한다. 이후 개선한다면 Vue3 전환 검토, API 계층 분리, 빌드/배포 자동화, Vue 쪽 공통 컴포넌트화 등을 고려할 수 있다.

요약

기존 JSP 프로젝트를 전면 교체하지 않고, Vue 프로젝트를 별도로 추가해 빌드 결과물만 기존 웹 애플리케이션의 **프론트엔드 배포 경로**에 연결해 **점진적 도입 사례**다. Vue2를 선택한 이유는 당시 운영 환경과 호환성 이슈 때문이다. 로그인 화면을 시작점으로 라우팅 구조를 구성했고 router, axios 통신을 중심으로 구성되었다. 형상관리에서는 `node_modules`와 빌드 결과물을 제외하고, 개발 시에는 `npm run serve`, 실행 시에는 Tomcat + 빌드 결과물을 사용하는 방식으로 정리했다.